

21st Century Alchemy

Part 2: Deep learning → very deep learning



Nicholas J. Cooper



Table of Contents

- Recognizing moving patterns
- Convolution and inductive bias
- Vanishing gradients and the 2nd quest for depth
- Residual networks

Last time...

- Perceptrons as a computational model of neurons
 - Birth of “learning by doing”
- Promising proof of concept results, but many limitations
 - Can’t learn XOR, parity, and other simple patterns
- Multilayer networks are much more *expressive*
 - Training such networks is harder
- Backpropagation -> 1st architecture explosion
 - Unlocked training deep neural networks

This Time...

- Pure generalization vs. weak generalization
 - AKA, where Perceptrons fail
- Digital image processing and convolution
 - Sliding filters for extracting features
- Convolutional neural networks and the 2nd quest for depth
 - What happens when we make networks **very** deep?
- Residual networks
 - Skip connections — “holes” in computational graphs

Moving Targets

Supervised Learning Terminology

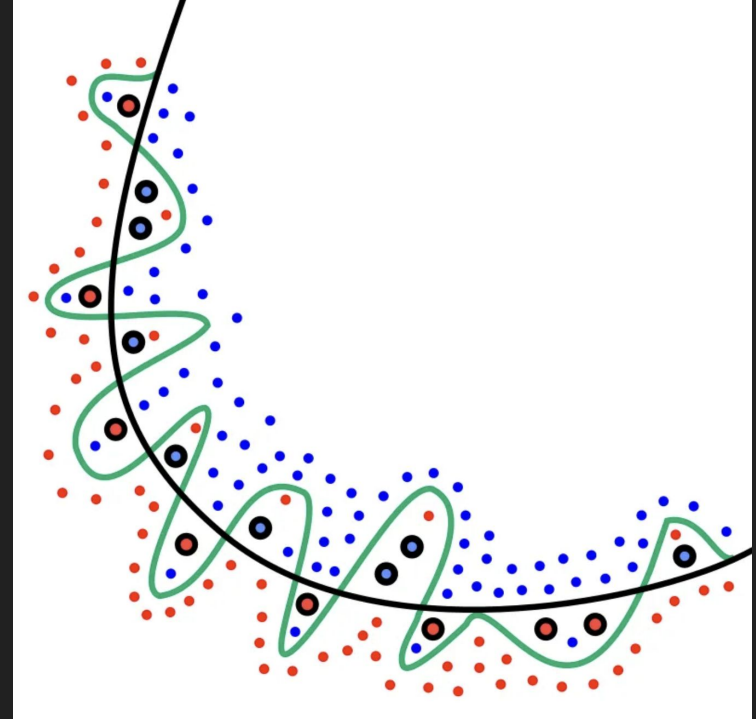
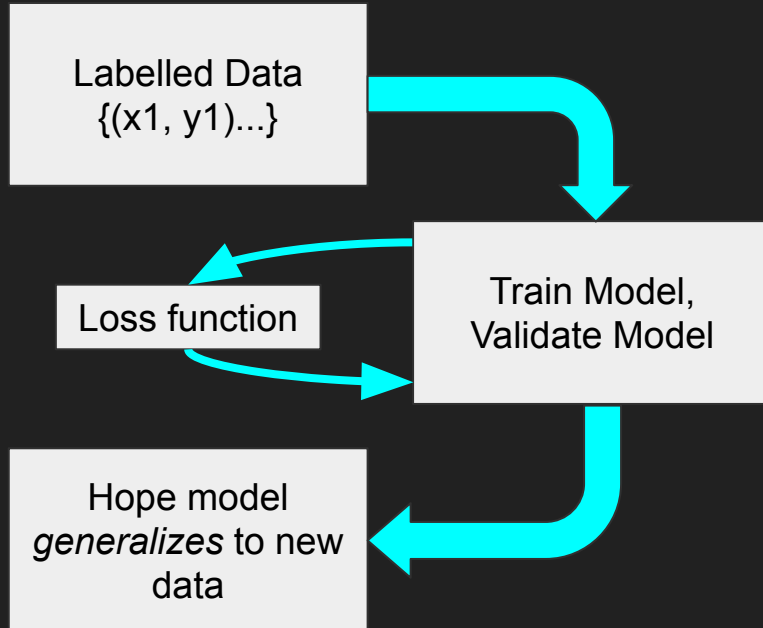


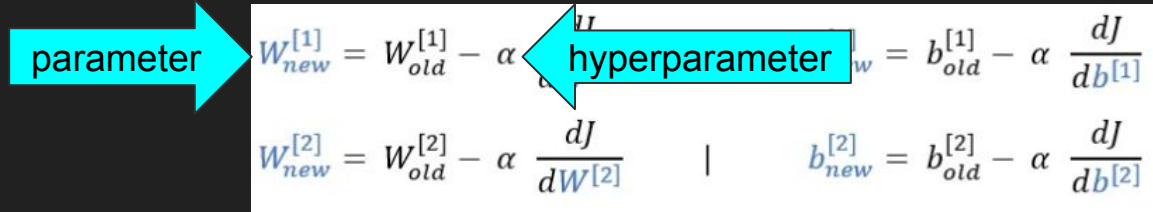
Figure taken from: <https://www.restack.io/>

Supervised Learning Terminology

- The collection of samples (x, y) is called the *training dataset*
- We use a *completely separate* collection of samples (x', y') as a *testing dataset*
 - It is very important to keep the training and testing samples *separate*
- We say that a learned model **generalizes** when it produces “good” predictions on the testing samples it has not seen during training
 - Typically, “good” means low average loss
 - Constructing models that generalize continues to be the ultimate goal of deep learning

General Machine Learning Terminology

- A *parameter* is a numeric value that is modified during model training
 - E.g., the weights and biases in a fully connected network
- A *hyperparameter* is a variable which controls the learning process
 - E.g., the learning rate



The diagram shows two equations for weight and bias updates. A red arrow labeled 'parameter' points to the weight $W_{new}^{[1]}$ in the first equation. A red arrow labeled 'hyperparameter' points to the learning rate α in both equations.

$$\begin{aligned} W_{new}^{[1]} &= W_{old}^{[1]} - \alpha \frac{dJ}{dW^{[1]}} & b_{new}^{[1]} &= b_{old}^{[1]} - \alpha \frac{dJ}{db^{[1]}} \\ W_{new}^{[2]} &= W_{old}^{[2]} - \alpha \frac{dJ}{dW^{[2]}} & b_{new}^{[2]} &= b_{old}^{[2]} - \alpha \frac{dJ}{db^{[2]}} \end{aligned}$$

- A training *epoch* is a collection of training iterations that cover the complete training dataset, i.e., the model is shown each available sample
 - Models are typically trained for *many* epochs (potentially takes days/years for full training)

Limits of Fully Connected Networks

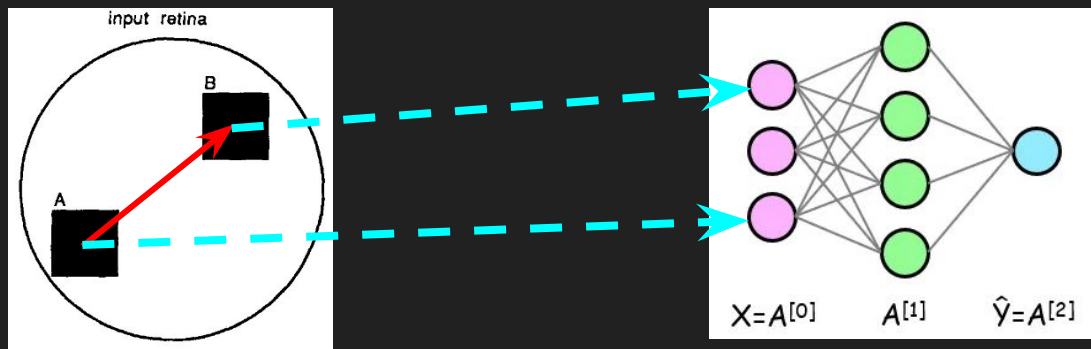
- **Recall:** the *universal approximation theorem* tells us that two layer FCNNs can learn any function, provided there are enough neurons in the hidden layer
- However, the theorem does *not* tell us how to find parameter values
 - It is an existence result, not a blueprint for finding low-loss solutions
- Frequently, 2 layer FCNNs will never find a good solution

The book [Minsky and Papert, 1969] does say that we don't think that there is an efficient way to make multilayer networks learn. Now, 'efficiently' in the 1960s meant a few thousand trials. Of course now if it does it in a million trials it is not so bad . . . There are two issues here. One is that one sense of 'efficiency' has changed. We don't care if it is a million now. (Minsky, interview)

Quote taken from: MIKEL OLAZARAN, "A Sociological History of the Neural Network Controversy", 1993

How to train complex, deep neural networks?

- This problem of finding “good” values for the parameters of DNNs “efficiently” continues to this day
 - Modern LLMs require *trillions* of training steps
- During the early stages of the 1st architecture explosion, recognizing shifting patterns proved to be too difficult for FCNNs to learn in practice



Quote taken from: MIKEL OLAZARAN, “A Sociological History of the Neural Network Controversy”, 1993

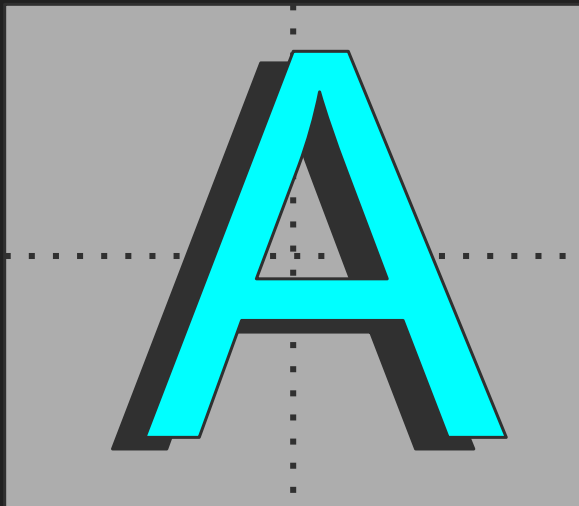
The Question of Generalization

- Early experiments with Perceptron and FCNNs studied “weak generalization”
 - Patterns used for testing *overlapped* with those used for training
 - Not really generalization in the modern sense
- Shifting patterns is a fundamental example of a “pure generalization” problem

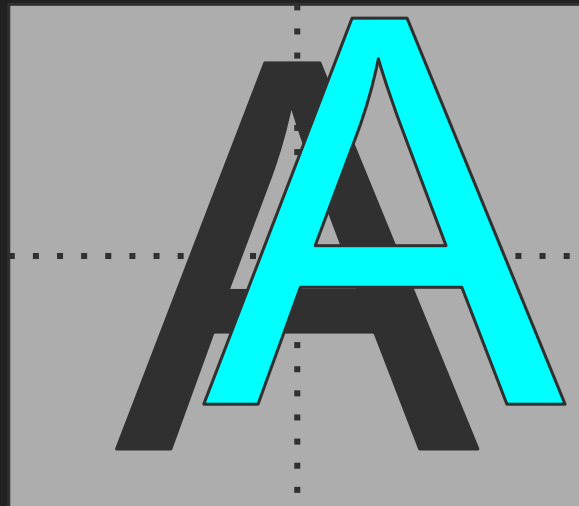
If the amount of overlap between the retinal areas occupied by two objects was big enough, then the perceptron would be able to classify them under the same category (otherwise it would not).⁶ Rosenblatt used the term “weak generalization” to refer to this type of overlap-based recognition, as opposed to “pure generalization,” which the elementary perceptron was *not* capable of.

Quote taken from: MIKEL OLAZARAN, “A Sociological History of the Neural Network Controversy”, 1993

Weak vs. Pure Generalization



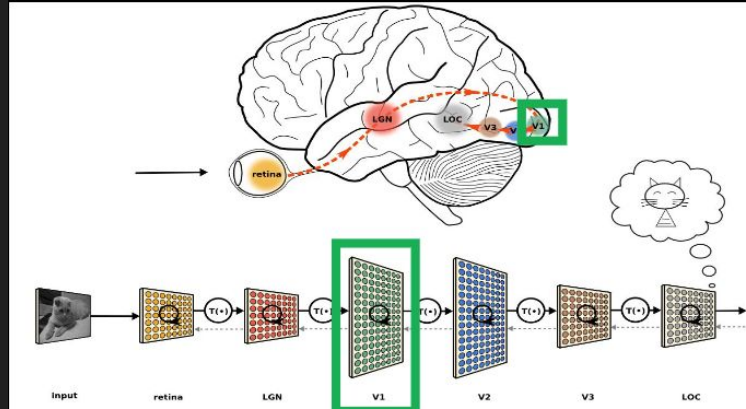
Weak Generalization



Pure Generalization

How to build networks for pure generalization?

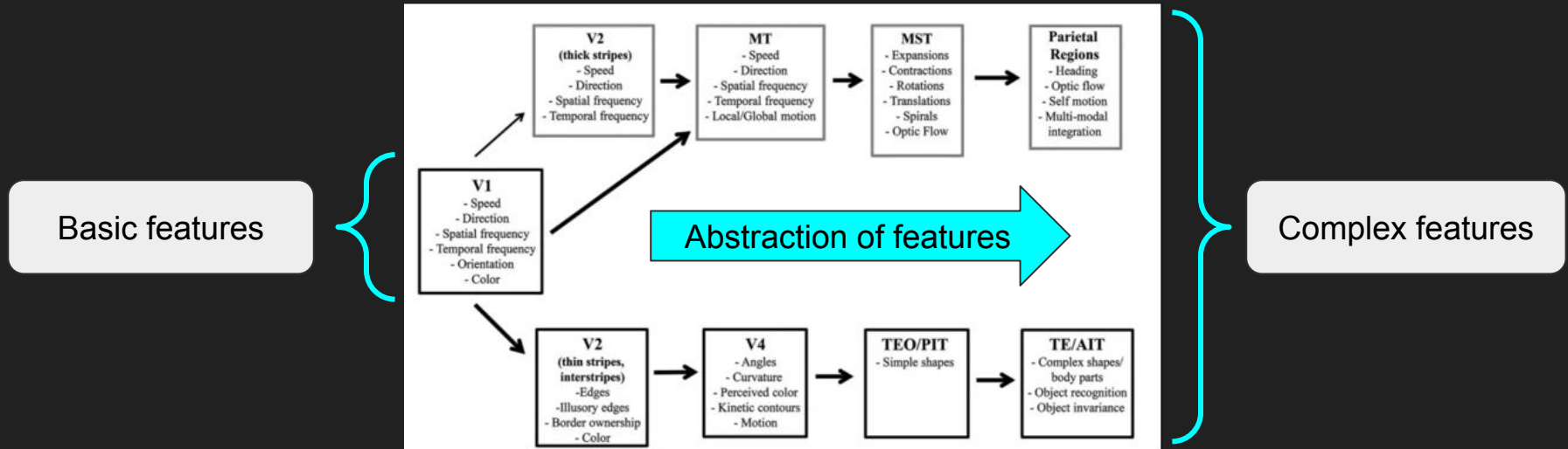
- Once again, start by drawing inspiration from natural biological systems
- In the 1960s, neurophysiologists David Hubel and Torsten Wiesel developed a (1981 Nobel prize winning) computational model of the visual cortex
 - Based on a *hierarchy* of neuron activation patterns



Picture credit: https://neuwritesd.files.wordpress.com/2015/10/visual_stream_small.png

Insights from Biology

- Less abstract features → more abstract features
- How to mimic this idea *architecturally*?

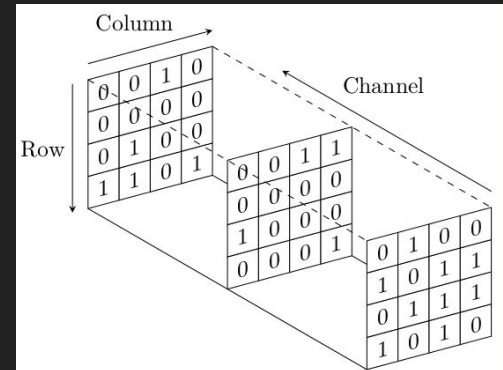


Picture credit: <https://www.o8t.com/blog/visual-system>

Convolutional Networks

Multidimensional Array Terminology

- Tensors are fascinating mathematical objects which are instrumental in deep learning via their ***multidimensional array*** representations
- Multidimensional arrays are powerful data structures that generalize the idea of matrix to any number of dimensions
- The i^{th} dimension of an array is called the i^{th} *axis*, or *mode*
- The dimensionality of an array is called its *order*

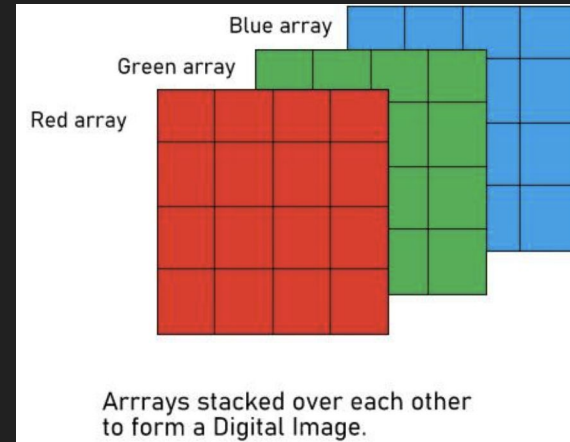


Picture credits: Mark Wibrow

Representing Images Digitally

- Images are usually encoded as order 3 multidimensional arrays
 - Shape (H, W, C) — height, width, channel
 - Entries in the array are **pixel values** — typically between 0 and 255 (8-bit color depth)
 - Color images use three channels (R, G, B)
 - Grayscale images use a single channel

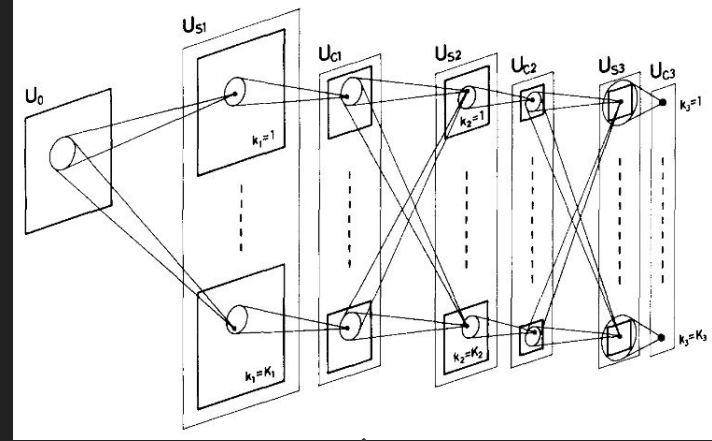
1	1	1	1	1	1	1	1	1	1
1	0	0	0	1	1	0	0	0	1
1	1	0	1	1	1	1	0	1	1
1	1	0	1	1	1	1	0	1	1
1	1	0	1	1	1	1	0	1	1
1	1	0	0	0	0	0	0	1	1
1	1	0	1	1	1	1	0	1	1
1	1	0	1	1	1	1	0	1	1
1	1	0	1	1	1	1	0	1	1
1	0	0	0	1	1	0	0	0	1
1	1	1	1	1	1	1	1	1	1



Picture credits: <http://preservationtutorial.library.cornell.edu/tutorial/intro/intro-01.html>;
<https://www.geeksforgeeks.org/software-engineering/matlab-rgb-image-representation/>

Convolutional Neural Networks (CNNs)

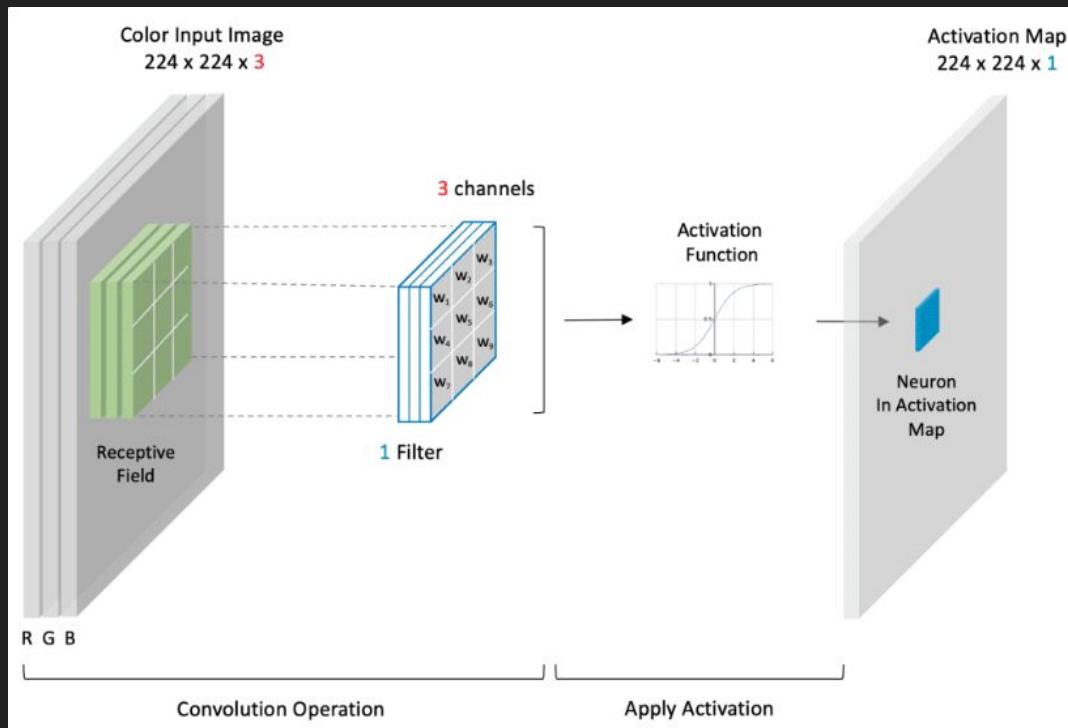
- One of the most important discoveries to come out of the 1st architecture explosion
- Key idea: use neural networks with *built in hierarchical representations*
- LeCun et. al., are credited with developing CNNs in 1989, but the core ideas can be traced back to at least 1980 in the work of Kunihiro Fukushima



Utilized a hierarchical network architecture, large->small activation patterns from left->right

Convolution Basics

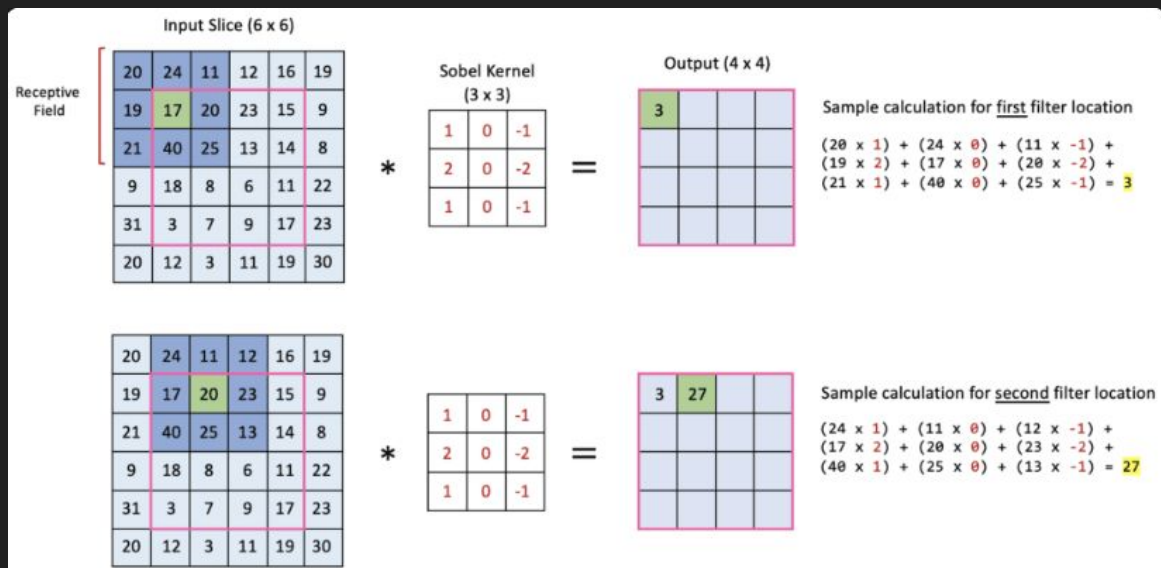
- Use *sliding filters* to scan input images
 - Filters are aka kernels
- For a computer, an image is an order 3 tensor (w,h,c)
 - Filters are also order 3
- Scan input for patterns with *local* dot product scores
 - Detect patterns even if they move!



Picture credit: <https://learnopencv.com/understanding-convolutional-neural-networks-cnn/>

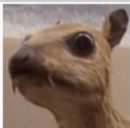
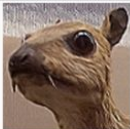
Convolution Basics

- Convolution produces output *feature maps*
 - Each location in the output indicates the *local response* from the associated *receptive field*



Picture credit: <https://learnopencv.com/understanding-convolutional-neural-networks-cnn/>

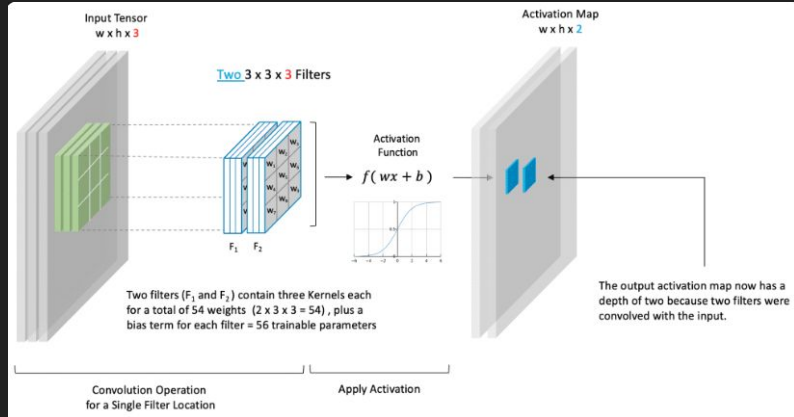
Convolution Examples

Operation	Filter	Convolved Image
Identity	$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$	
Edge Detect	$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$	
Sharpen	$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$	
Box blur (normalized)	$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$	
Gaussian blur (approximation)	$\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$	

Picture credit: <https://ujjwalkarn.me/2016/08/11/intuitive-explanation-convnets/>

The 2-d Convolution Layer

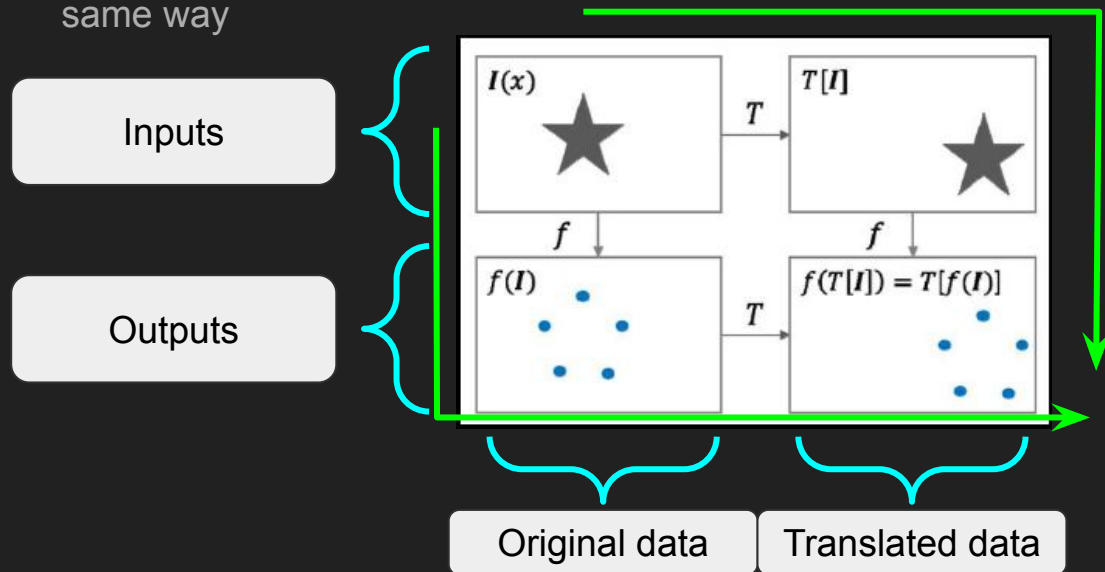
- Multilayer perceptrons are built from fully connected layers
 - Computationally, each FC layer is a learned matrix multiplication
- CNNs are built from 2-d convolution layers
 - Computationally, each 2-d conv. layer is a collection of learned convolutions with many different filters



Picture credit: <https://learnopencv.com/understanding-convolutional-neural-networks-cnn/>

Convolution Layers are Translation Equivariant!

- Unlike fully connected layers, convolutional layers have a very nice property
 - **Translation equivariance** — if the input patterns move, the output feature maps move in the same way



Picture credit: <https://www.mdpi.com/2076-3417/10/9/3161>

Convolution's Inductive Bias

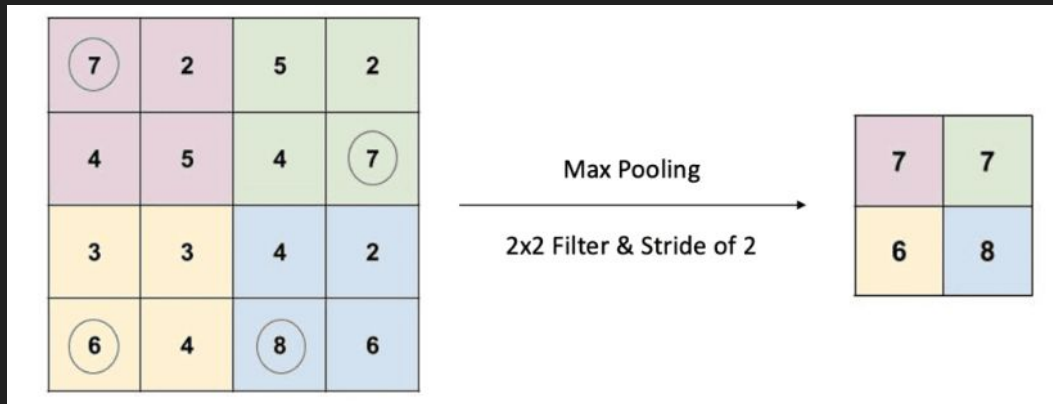
- Typically, there are *many* different low-loss points in parameter space
- **Inductive biases** are the tendencies of an architecture to find particular types of solutions during the training process
- Convolution's translation equivariance is a particular type of inductive bias
- By construction, a convolutional layer cannot learn a solution (linear transformation) which is not translation equivariant
 - In other words, a 2-d conv. layer can be thought of as a particular “sub-layer” of a fully connected layer (linear transformations w/o any restrictions)

Pooling Layers

- We need one more ingredient to construct CNNs
- Typically, convolution layers do not change the *shape* of the input tensors
 - For example, a 64x64 image produces a 64x64 feature map
 - Other *hyperparameters*, e.g., stride, padding, allow conv. layers to change input shape
- As CNNs aim to mimic the hierarchical structure of biological visual systems, *pooling layers* are required
- Idea: downsample the feature maps in a differentiable manner
- Pooling layers define the **stages** of a CNN

Pooling Layers

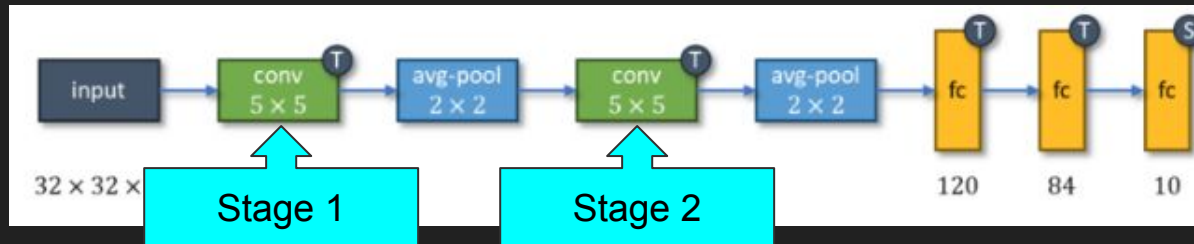
- Partition each *channel* of a feature map into rectangular regions
- Select only the largest activation value from each region
 - This is called max pooling, other variants exist, e.g., average pooling
- Use these selected values to form a new, smaller feature map



Picture credit: <https://learnopencv.com/understanding-convolutional-neural-networks-cnn/>

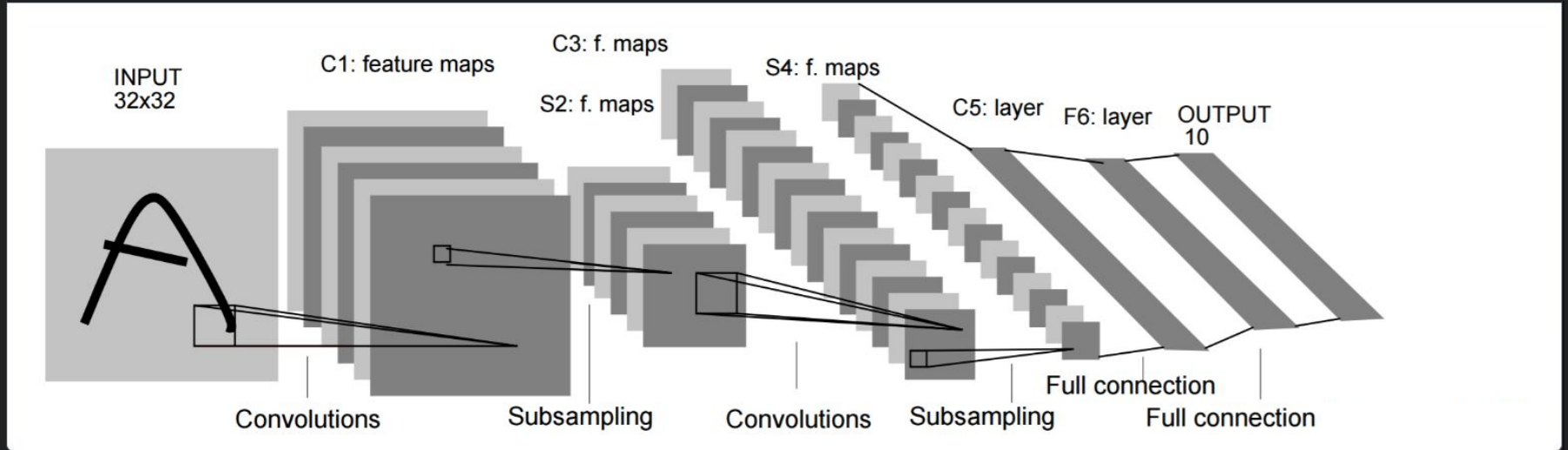
Finally, a CNN!

- The highly influential LeNet architecture was used in LeCun et. al. in their 1989 paper to classify handwritten digits (MNIST dataset)
- Composed of ~60k parameters, it used a combination of convolution, pooling, and fully connected layers
- After each learned linear transformation, a non-linear activation is applied



Picture credit: <https://towardsdatascience.com/5-most-well-known-cnn-architectures-visualized-af76f1f0065e/>

LeNet Visualized



Picture credit: <https://github.com/LukaszObara/LeNet5/blob/master/images/architec2.png>

Recognizing Moving Patterns

- The MNIST dataset contains thousands of images of handwritten digits
- Unlike the early perceptron experiments, this is a pure generalization problem
- LeNet achieved ~99% accuracy on the test set and beyond!



Figure 2: Examples of normalized digits from the testing set.

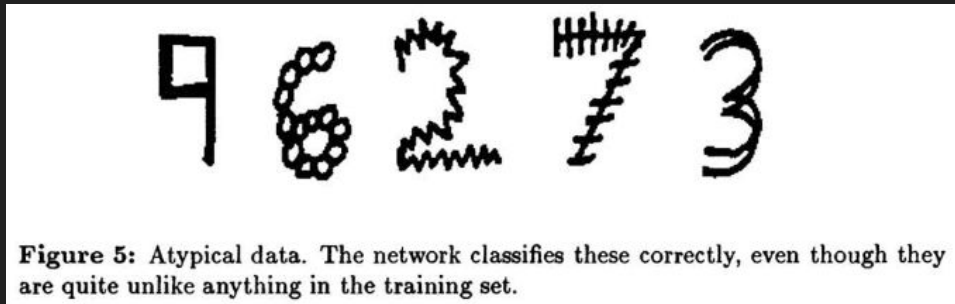


Figure 5: Atypical data. The network classifies these correctly, even though they are quite unlike anything in the training set.

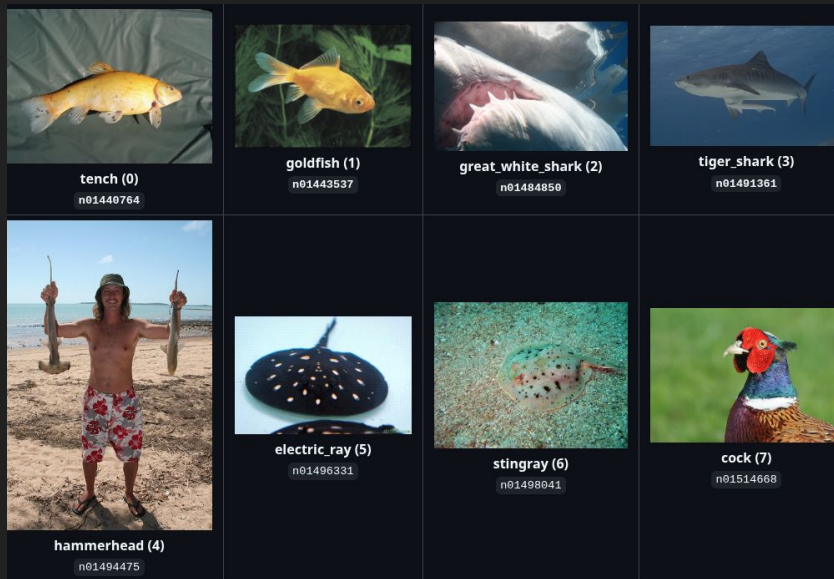
Picture credit: LeCun et. al., "Handwritten Digit Recognition with a Back-Propagation Network", NeurIPS, 1989

Deeper models for bigger datasets

- LeNet launched the first era of AI scaling as researchers extended CNNs to more complex pattern recognition tasks
- Recognizing digits is cool, but what about “real” visual understanding?
 - Many new datasets were introduced, e.g., [CIFAR](#), [ImageNet](#), [COCO](#).
- As tasks and their associated datasets became more demanding, architectures became deeper
 - Necessary to ensure sufficient model expressivity
- During this 1st architecture explosion, data and models scaled to millions of samples and parameters

The ImageNet Dataset

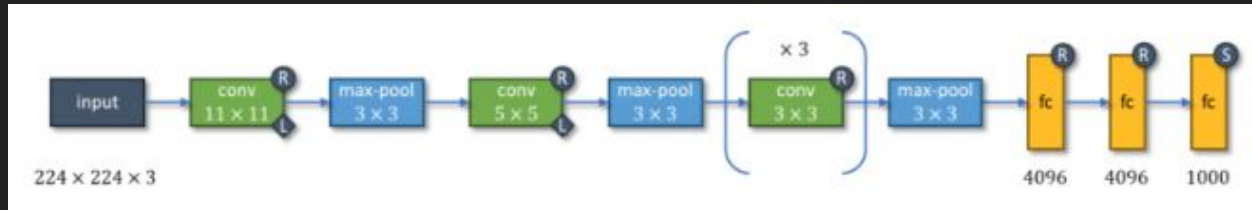
- ImageNet contains ~1.2 *million* natural images from 1000 classes
 - Still considered a difficult dataset to this day



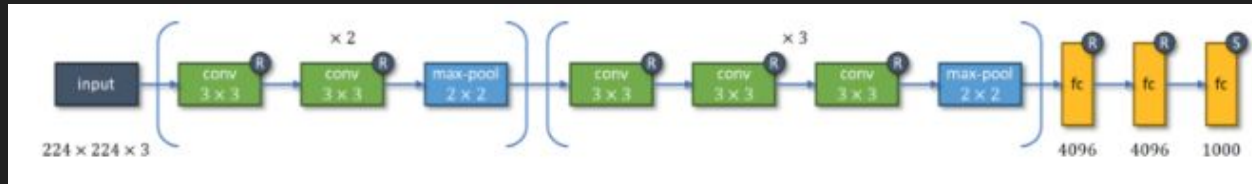
Picture credit: <https://github.com/EliSchwartz/imagenet-sample-images/blob/master/gallery.md>

Highlights from the Architecture Zoo

- AlexNet and VGG are very important architectures from the 1st explosion
 - The first models to achieve good performance on ImageNet
- AlexNet was introduced in 2012; used ~60 million parameters and 8 layers



- VGG was introduced in 2014; used ~140 million parameters and 19 layers



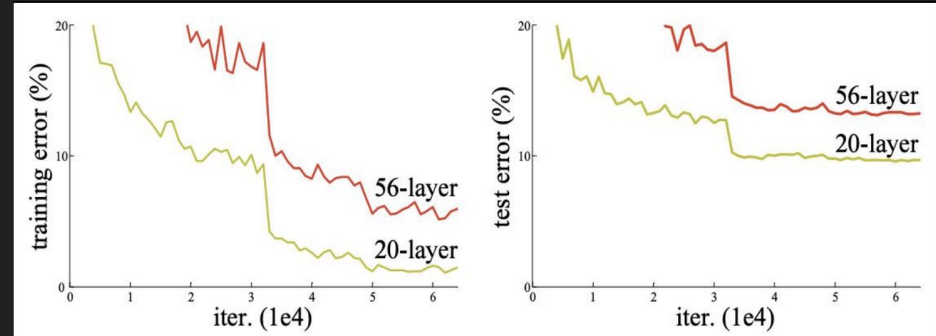
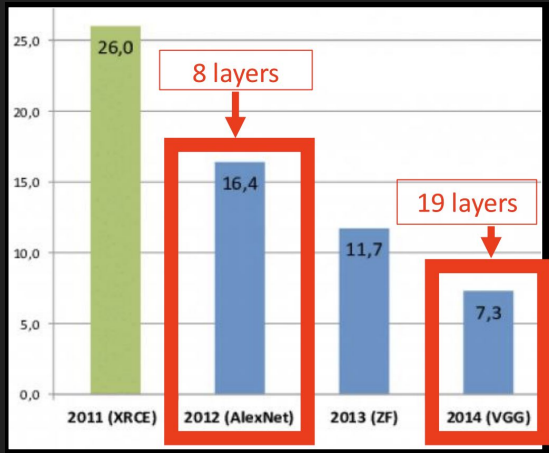
Picture credit: <https://towardsdatascience.com/5-most-well-known-cnn-architectures-visualized-af76f1f0065e/>

What does a deep CNN “see”?

- Demo (code: https://github.com/combinatoriallabs/SEMF_26_Demos/)

A Mystery: Deeper is Worse?

- For a while, making architectures deeper worked really well
- Then, some puzzling behavior emerged: very deep networks perform worse?



Picture credits: <https://www.edge-ai-vision.com/2018/07/deep-learning-in-five-and-a-half-minutes/>
He et. al., "Deep Residual Learning for Image Recognition", 2016

The 2nd Quest for Depth

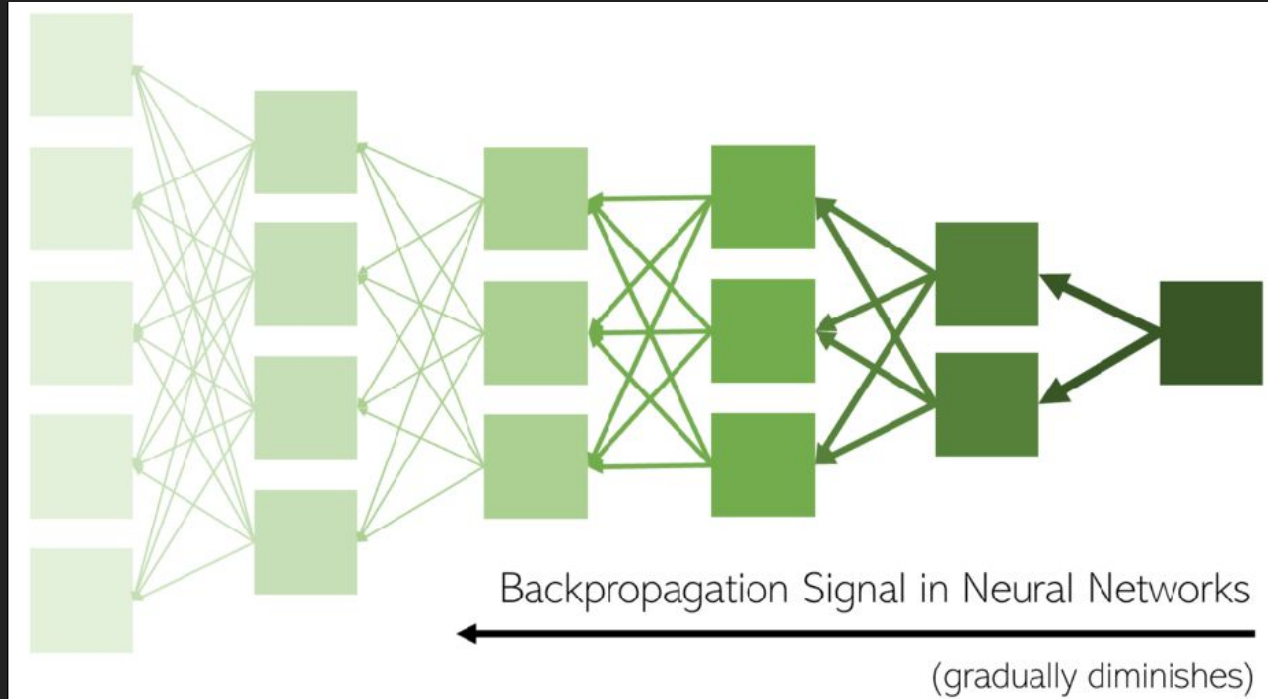
A Problem: Vanishing Gradients

- As exploration extended into uncharted territory of deeper models with many types of layers, a new problem emerged
- Recall that backpropagation boils down to sequence of *multiplications*
- As soon as *any* of the terms in the backprop equations become small, the parameter gradients disappear!

$$\frac{\partial J}{\partial W^{[1]}} = \frac{\partial J}{\partial A^{[2]}} \frac{\partial A^{[2]}}{\partial Z^{[2]}} \frac{\partial Z^{[2]}}{\partial A^{[1]}} \frac{\partial A^{[1]}}{\partial Z^{[1]}} \frac{\partial Z^{[1]}}{\partial W^{[1]}} = (A^{[2]} - Y) \cdot A^{[2]}(1 - A^{[2]}) \cdot W^{[2]} \cdot A^{[1]}(1 - A^{[1]}) \cdot X$$

If any of these terms approach zero, so does the product

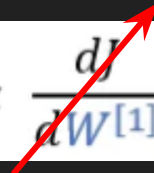
Vanishing Gradients Visualized



Picture credits: <https://towardsdatascience.com/batch-normalization-the-greatest-breakthrough-in-deep-learning-77e64909d81d>

Vanishing/Exploding Gradients Prevent Depth

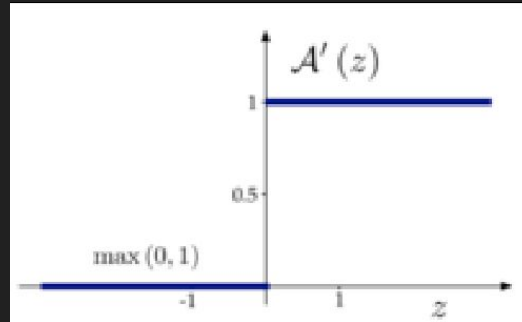
- There is a dual to vanishing gradients, *exploding gradients*
 - Occurs when one or more terms in the backprop equation becomes very large
- Unfortunately, as we make models deeper, we introduce more and more terms to the backprop equations
- In other words, deeper models are more prone to the vanishing/exploding gradient problem
- Together, vanishing/exploding gradients became a major limitation in the quest for deeper, more powerful architectures
 - If model parameters cannot be updated, learning stops!


$$W_{new}^{[1]} = W_{old}^{[1]} - \alpha \frac{dJ}{dW^{[1]}}$$

Picture credits: <https://medium.com/data-science/neural-networks-backpropagation-by-dr-lihi-gur-arie-27be67d8fdce>

A Suspect: Activation Functions

- **Recall:** neural networks require *non-linear* activation functions
 - Otherwise, they could only express linear functions
- The activation used in the original perceptron was a simple step function
 - Gradient is “always” zero!



Picture credits: Masi et al. Journal of the Mechanics and Physics of Solids. 2021

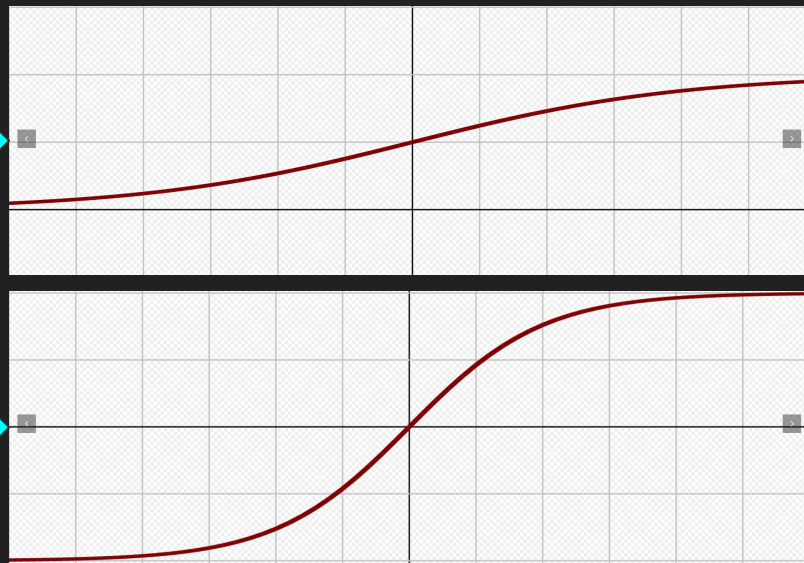
Tanh and Sigmoid Activations

- Architectures from the 1st explosion used a variety of activations such as tanh and sigmoid
- What's the problem here?

Gradients of both approach zero as the input magnitude increases!

Sigmoid

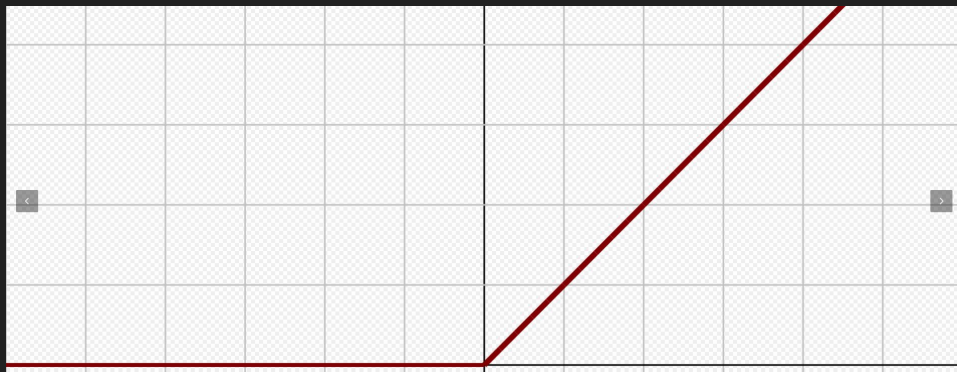
Tanh



Picture credits: https://en.wikipedia.org/wiki/Activation_function

The ReLU Activation Function

- Idea: use a **non-saturating** function to prevent introducing small gradient terms when backpropagating through non-linear activations
- The rectified linear unit (ReLU) function is the standard example of such a function
 - Defined by $\text{ReLU}(x) = \max(0, x)$

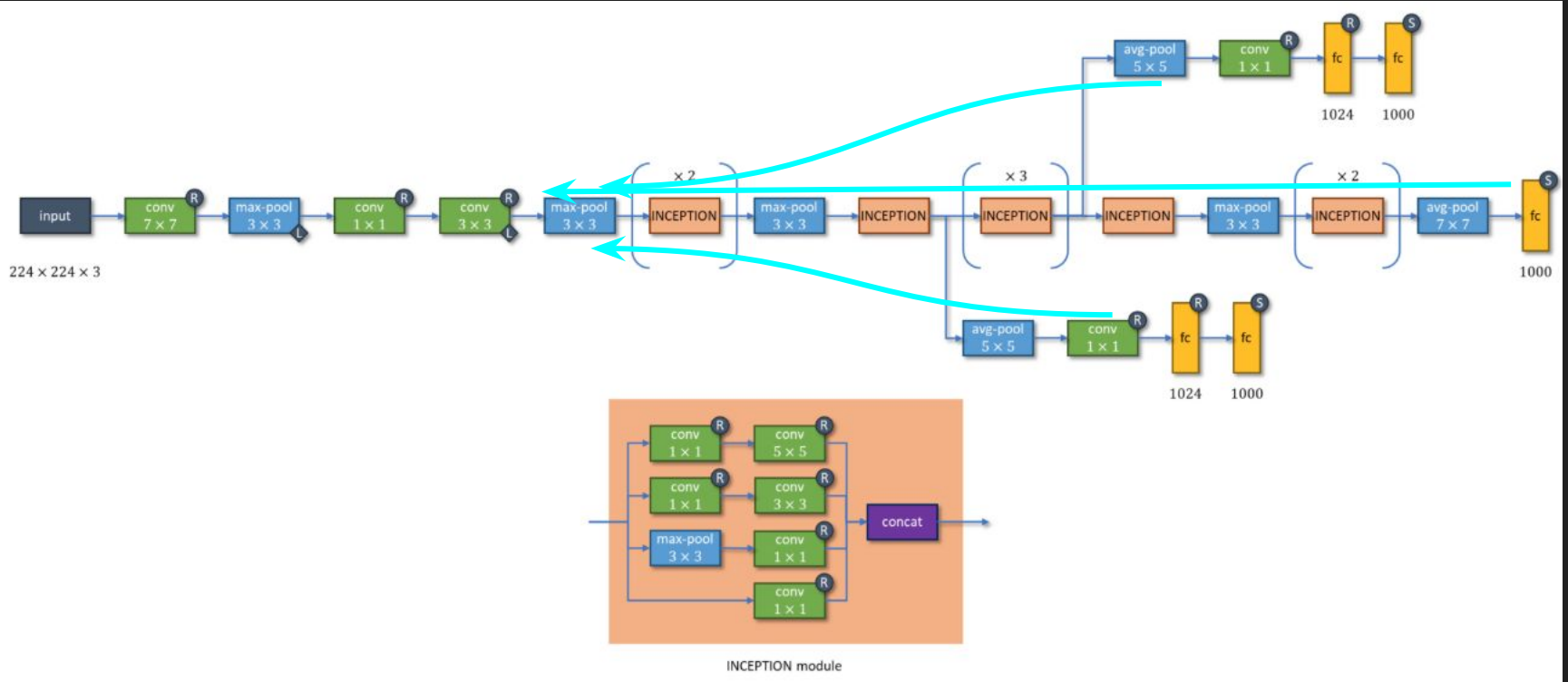


Picture credits: https://en.wikipedia.org/wiki/Activation_function

Vanishing Gradients Remain

- ReLU (and its variants, e.g., Leaky ReLU, GeLU) help with vanishing gradients but do not completely eliminate the problem
 - Gradients are still small when inputs are negative
- Many attempts were made to solve vanishing gradients *architecturally*
- Inception V1 used a “trident” structure to inject additional backprop signals to the early layers

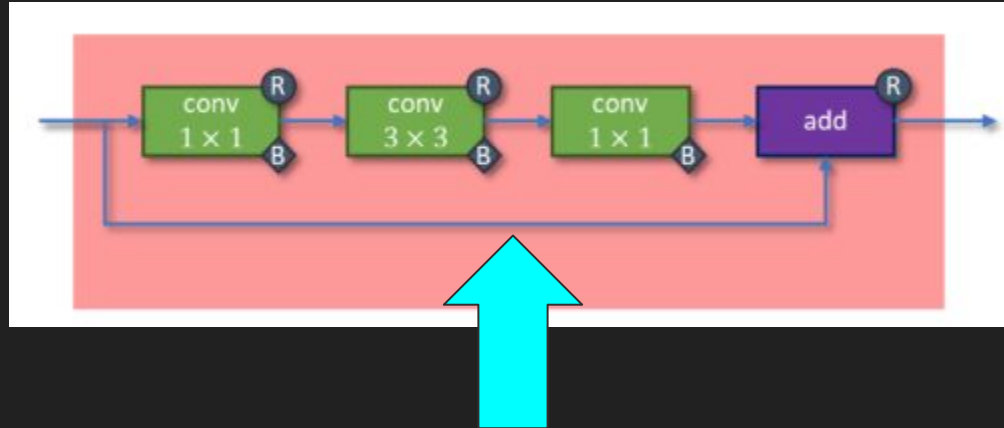
Inception V1



Picture credit: <https://towardsdatascience.com/5-most-well-known-cnn-architectures-visualized-af76f1f0065e/>

Breakthrough: Residual Networks

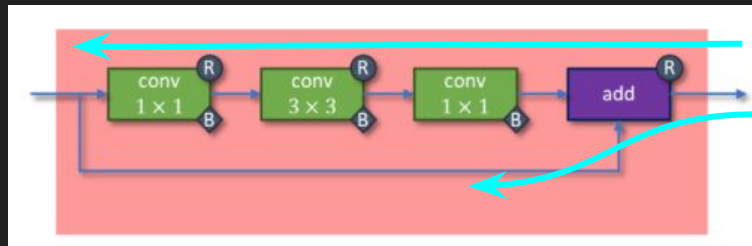
- Inception's "trident" structure found some success, but ultimately, a key breakthrough on the vanishing/exploding gradient problem came in 2016
- Residual networks, or ResNets for short, introduced *skip connections*



Picture credit: <https://towardsdatascience.com/5-most-well-known-cnn-architectures-visualized-af76f1f0065e/>

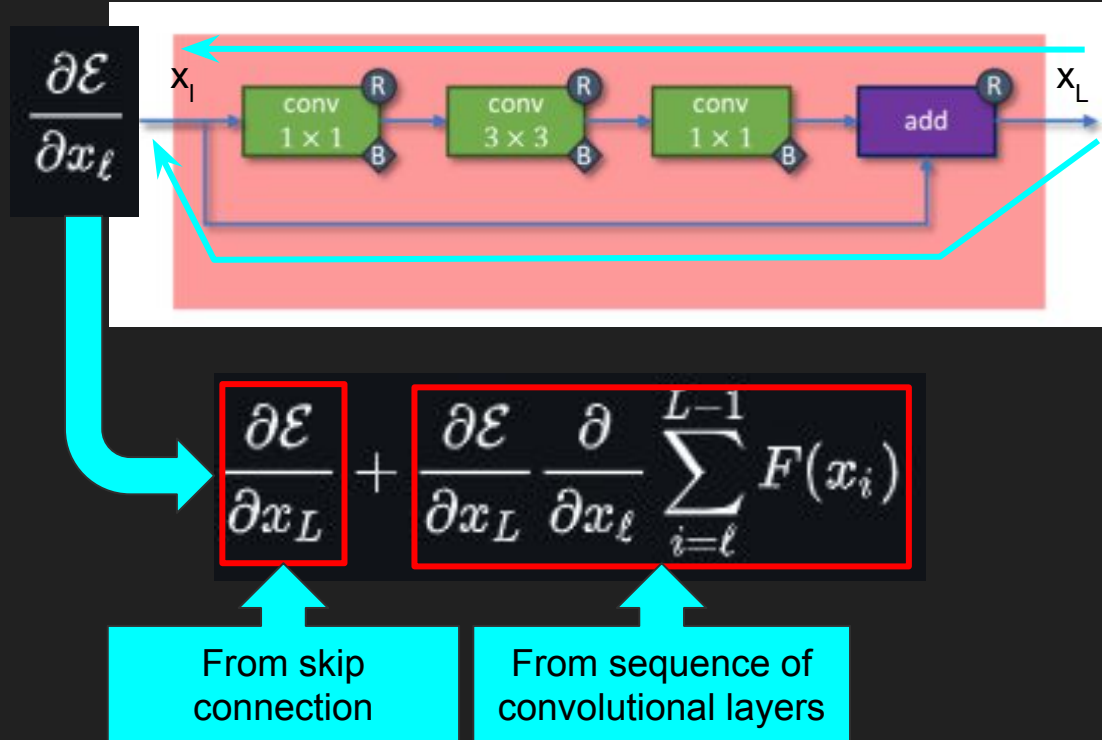
Skip Connections

- Idea: short circuit the input around several convolutional layers with element-wise addition
- Effectively, this forces the convolutional layers to learn *offsets* instead of entire *transformations*
- Critically, skip connections mitigate vanishing gradients by introducing two *parallel paths* in the computational graph



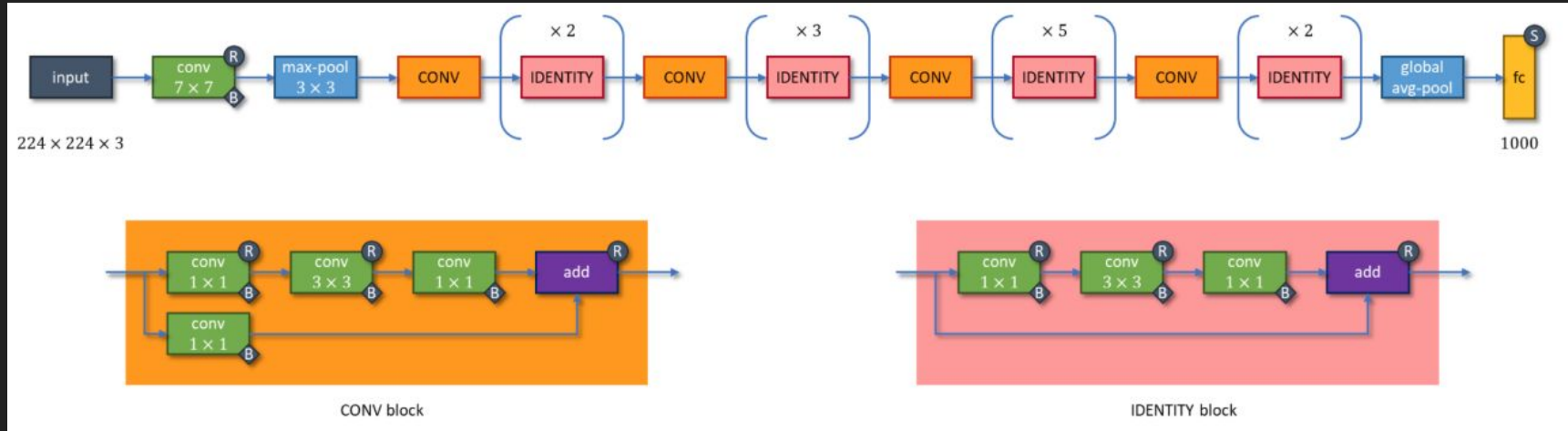
Picture credit: <https://towardsdatascience.com/5-most-well-known-cnn-architectures-visualized-af76f1f0065e/>

Skip Connection Gradients



A Complete ResNet Architecture

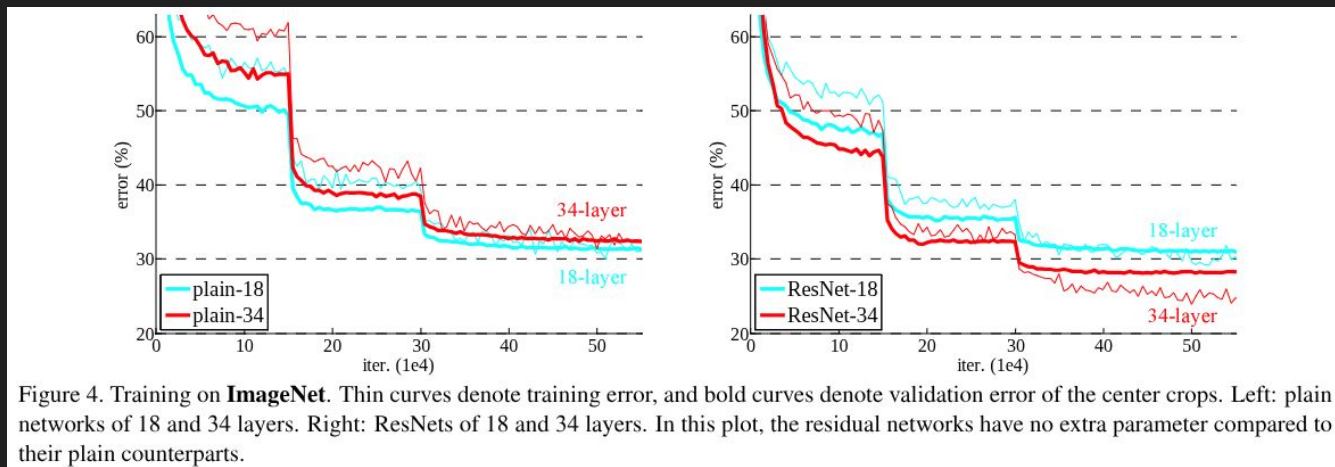
- ResNet-50 has ~26 million params and 50 layers
- Used two types of residual blocks



Picture credit: <https://towardsdatascience.com/5-most-well-known-cnn-architectures-visualized-af76f1f0065e/>

Impact of Skip Connections: Deeper is better again!

ResNet-34 B	21.84	5.71
ResNet-34 C	21.53	5.60
ResNet-50	20.74	5.25
ResNet-101	19.87	4.60
ResNet-152	19.38	4.49



Picture credit: He et. al., “Deep Residual Learning for Image Recognition”, 2016

Impact of Skip Connections

- Skip connections have proved to be an immensely powerful architectural motif
- Unlocked building *very deep* networks
- They are used in most modern deep neural networks, including LLMs
- The primary discovery to come from the 2nd quest for depth

Summary

- Perceptrons struggle with moving patterns
- Convolutional layers excel at recognizing shifting patterns
 - Translation equivariance
- LeNet inspired the development of many types of CNNs
- Shortly after, deep learning hits its second depth challenge
 - Vanishing/exploding gradients prevent very deep networks from learning
- ResNet introduced skip connections
 - Mitigate vanishing gradients

Coming Up...

- Basics of digital language processing
- Recurrent neural networks (RNNs)
 - Duality between ResNets and RNNs
 - The vanishing context problem
- Attention and context skip connections
- Self attention, transformer blocks, and the transformer architecture
- Recent theories of deep neural networks
 - Statistics, neuro-manifolds, and monad algebra homomorphisms, oh my!